Project Title

Loan Approval Prediction using Random Forest Classification with Model Comparison

Introduction (Write This)

- Random Forest is an ensemble learning algorithm that improves the performance of Decision Trees by combining multiple trees and reducing overfitting.
- In this project, Random Forest Classification is applied to predict loan approval status and compared with Decision Tree and Logistic Regression models.

Problem Statement

The objective is to predict whether a loan application will be approved or not based on applicant details using Random Forest Classification and evaluate its performance against previously implemented models.

Build Random Forest Classifier

- Reduce overfitting seen in Decision Tree
- Improve accuracy and stability
- Compare with:
  - Logistic Regression
  - Decision Tree

STEP 1 : Preprocessing

1.1 Import Libraries

In [2]:
```
import pandas as pd
import numpy as np
```

1.2 Load Dataset

In [3]:
```
df = pd.read_csv("Loan_dataset.csv")
```

1.3 Check and Handle Missing Values

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
Unnamed: 0            0
Loan_ID               0
Gender               10
Married               1
Dependents            9
Education             0
Self_Employed        29
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           16
Loan_Amount_Term     13
Credit_History       43
Property_Area         0
Loan_Status           0
dtype: int64
```

In [5]:
```
# Categorical → mode
df['Gender']= df['Gender'].fillna(df['Gender'].mode()[0])
df['Married']= df['Married'].fillna(df['Married'].mode()[0])
df['Dependents']= df['Dependents'].fillna(df['Dependents'].mode()[0])
df['Self_Employed']= df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])

# Numerical → median
df['LoanAmount']= df['LoanAmount'].fillna(df['LoanAmount'].median())
df['Loan_Amount_Term']= df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].med

# Important feature
df['Credit_History']= df['Credit_History'].fillna(df['Credit_History'].mode()[0]
```

1.4 Drop Unnecessary Column

In [ ]:
```
df.drop("Loan_ID", axis=1, inplace=True)
```

1.5 Encode Categorical Features

In [9]:
```
df = pd.get_dummies(df, drop_first=True)
```

1.6 Feature and Target

In [10]:
```
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

1.7 Train-Test-Split

In [11]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

STEP 2: Model Building

2.1 Import Model

In [12]:
```
from sklearn.ensemble import RandomForestClassifier
```

2.2 Create Model

In [13]:
```
rf_clf = RandomForestClassifier(random_state=42)
```

2.3 Train Model

In [14]:
```
rf_clf.fit(X_train, y_train)
```

Out[14]:
```
▾ RandomForestClassifier
▸ Parameters
```

2.4 Prediction

In [16]:
```
y_pred_rf = rf_clf.predict(X_test)
```

2.5 Evalute Model

In [17]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Precision:", precision_score(y_test, y_pred_rf))
print("Recall:", recall_score(y_test, y_pred_rf))
print("F1 Score:", f1_score(y_test, y_pred_rf))
```

```
Accuracy: 0.797979797979798
Precision: 0.7922077922077922
Recall: 0.9384615384615385
F1 Score: 0.8591549295774648
```

2.6 Check Overfitting

In [19]:
```
train_pred_rf = rf_clf.predict(X_train)

print("Training Accuracy:", accuracy_score(y_train, train_pred_rf))
print("Testing Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Training Accuracy: 1.0
Testing Accuracy: 0.797979797979798
```

Model Performance (Before Tuning)

- Training Accuracy: 1.0
- Testing Accuracy: 0.80

Interpretation

The Random Forest model achieved perfect accuracy on the training dataset, indicating that it has learned the training data completely.

However, the testing accuracy is significantly lower, which suggests that the model is overfitting and not generalizing well to unseen data.

Overfitting Analysis

The model shows a large gap between training and testing accuracy This indicates that the trees are too complex The model is capturing noise in the training data

Key Insight

Although Random Forest generally reduces overfitting compared to Decision Trees, it can still overfit if the trees are too deep or if the dataset is small.

STEP 3: Hyperparameter Tuning

Objective

- Reduce overfitting
- Improve testing accuracy
- Build a more stable model

3.1 Import

In [20]:
```
from sklearn.model_selection import GridSearchCV
```

3.2 Parameter Grid

In [26]:
```
param_grid = {
    "n_estimators":[50,100,200],
    "max_depth":[3,5,7,10],
    "min_samples_split":[2,5,10],
    "min_samples_leaf":[1,2,4]
}
```

3.3 Apply Grid Search

In [27]:
```
grid_rf = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_rf.fit(X_train, y_train)
```

Out[27]:
```
▸ GridSearchCV
▸ best_estimator_: RandomForestClassifier
    ▸ RandomForestClassifier
```

3.4 Best Parameters

In [28]:
```
print("Best Parameters:", grid_rf.best_params_)
```

```
Best Parameters: {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 10,
'n_estimators': 200}
```

3.5 Best Model

In [29]:
```
best_rf = grid_rf.best_estimator_
```

3.6 Prediction

In [31]:
```
y_pred_rf_tuned = best_rf.predict(X_test)
```

3.7 Evaluation

In [32]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

print("Accuracy:", accuracy_score(y_test, y_pred_rf_tuned))
print("Precision:", precision_score(y_test, y_pred_rf_tuned))
print("Recall:", recall_score(y_test, y_pred_rf_tuned))
print("F1 Score:", f1_score(y_test, y_pred_rf_tuned))
```

```
Accuracy: 0.8282828282828283
Precision: 0.7926829268292683
Recall: 1.0
F1 Score: 0.8843537414965986
```

3.8 Overfitting Check (After Tuning)

In [33]:
```
train_pred_rf_tuned = best_rf.predict(X_train)

print("Training Accuracy (Tuned):", accuracy_score(y_train, train_pred_rf_tuned)
print("Testing Accuracy (Tuned):", accuracy_score(y_test, y_pred_rf_tuned))
```

```
Training Accuracy (Tuned): 0.8188775510204082
Testing Accuracy (Tuned): 0.8282828282828283
```

Model Comparison

Key Comparison

- Random Forest and Logistic Regression achieved similar accuracy (~0.83)
- Decision Tree performed slightly lower (~0.82)
- Random Forest improved over Decision Tree by reducing overfitting
- Logistic Regression remained stable across all cases

Conclusion

Random Forest provided improved performance over Decision Tree by reducing overfitting and achieving stable accuracy. However, its performance was similar to Logistic Regression, indicating that the dataset is relatively simple and can be effectively

handled by linear models. Therefore, both Logistic Regression and Random Forest are suitable, with Random Forest being more robust.